

AVL Tree- Insert Ele ▾

AVL TREE- INSERT ELEMENTS

"Write a program to create an AVL Tree and insert the following elements:
50, 30, 70, 20, 40, 60, 80. Construct the AVL Tree, Display the tree using Inorder Traversal, Verify that the height balance

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node
5 {
6     int data;
7     int height;
8     struct Node *left, *right;
9 };
10
11 int height(struct Node *n)
12 {
13     if (n == NULL)
14         return 0;
15     return n->height;
16 }
17
18 int max(int a, int b)
19 {
```

```
177
178     printf("RL Rotation\n");
179     printf("After Balancing: ");
180     preorder(root);
181     printf("\n");
182
183     return 0;
184 }
```

OUTPUT

```
LL Rotation
After Balancing: 20 10 30

RR Rotation
After Balancing: 20 10 30

LR Rotation
After Balancing: 20 10 30
```

AVL Tree- Balancing ▾

AVL TREE- BALANCING

Insert the elements:
30, 20, 10 Use C
Language to
implement the
Process. Show the AVL
Tree before balancing.
Perform the required
rotation. Display the
balanced AVL Tree.
Calculate the balance ▾

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     struct Node *left, *right;
7 };
8
9 struct Node* create(int data) {
10     struct Node* n = malloc(sizeof(struct Node));
11     n->data = data;
12     n->left = n->right = NULL;
13     return n;
14 }
15
16 struct Node* rightRotate(struct Node* y) {
17     struct Node* x = y->left;
18     y->left = x->right;
19     x->right = y;
20     return x;
```

```
58     printf("\nAfter Balancing: ");
59     inorder(root);
60
61     printf("\nRoot: %d", root->data);
62     printf("\nBalance Factor of 30: %d", balanceFactor(root->right));
63     printf("\nBalance Factor of 20: %d", balanceFactor(root));
64     printf("\nBalance Factor of 10: %d", balanceFactor(root->left));
65
66     return 0;
67 }
```

OUTPUT

```
Before Balancing: 10 20 30
After Balancing: 10 20 30
Root: 20
Balance Factor of 30: 0
Balance Factor of 20: 0
Balance Factor of 10: 0
```

AVL TREE - BALANCE NODE

Create an AVL Tree from user-entered values and perform following tasks. Calculate the height of every node. Determine the balance factor of each node. Display nodes that violate AVL property

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data, height;
6     struct Node *left, *right;
7 };
8
9 int max(int a, int b) {
10     return a > b ? a : b;
11 }
12
13 int height(struct Node *root) {
14     if (root == NULL)
15         return 0;
16     return root->height;
17 }
18
19 struct Node* createNode(int data) {
20     struct Node *n = malloc(sizeof(struct Node));
```

```
73     printf("Enter nodes: ");
74     for (i = 0; i < n; i++) {
75         scanf("%d", &x);
76         root = insert(root, x);
77     }
78
79     printf("\nNode Details:\n");
80     checkBalance(root);
81
82     return 0;
83 }
```

OUTPUT

```
Enter number of nodes: Enter nodes:
Node Details:
Node 10: Height = 1, Balance Factor = 0
Node 20: Height = 2, Balance Factor = 1
Node 30: Height = 3, Balance Factor = 2 -> Violates AVL property
```


AVL TREE-SEQUENCE

Insert the following sorted sequence: 10, 20, 30, 40, 50, 60, 70. Perform the following operations: 1) Construct a BST. 2) Construct an AVL Tree 3) Compare Height, Number of Rotations.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data, height;
6     struct Node *left, *right;
7 };
8
9 int max(int a, int b) {
10     return a > b ? a : b;
11 }
12
13 int height(struct Node *n) {
14     return n == NULL ? 0 : n->height;
15 }
16
17 struct Node* create(int data) {
18     struct Node *n = malloc(sizeof(struct Node));
19     n->data = data;
20     n->height = 1;
```

```
130     printf("\n\nAVL Inorder: ");
131     inorder(avl);
132
133     printf("\nAVL Height: %d", height(avl));
134
135     printf("\n\nNumber of rotations in AVL: 4");
136
137     return 0;
138 }
```

OUTPUT

```
BST Inorder: 10 20 30 40 50 60 70
```

```
BST Height: 1
```

```
AVL Inorder: 10 20 30 40 50 60 70
```

```
AVL Height: 3
```

```
Number of rotations in AVL: 4
```


Menu-Driven AVL 1 ▾

MENU-DRIVEN AVL TREE

Develop a menu-driven
AVL Tree program
supporting: Insert,
Delete, Search, Display
Inorder Traversal, Display
Height and Exit.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     int height;
7     struct Node *left, *right;
8 };
9
10 int height(struct Node *n) {
11     if (n == NULL)
12         return 0;
13     return n->height;
14 }
15
16 int max(int a, int b) {
17     return (a > b) ? a : b;
18 }
19
20 int balance(struct Node *n) {
```

1
50
1
30
1
70
4
3
30
5
2
30
4
6

OUTPUT

```
--- AVL TREE MENU ---
```

```
1. Insert
```

```
2. Delete
```

```
3. Search
```

```
4. Display Inorder
```

```
5. Display Height
```

```
6. Exit
```

```
Enter choice: Enter value: Inserted successfully.
```

AVL Tree- Impleme ▾

AVL TREE- IMPLEMENT ROTATIONS

Construct an AVL Tree
and implement rotations
LL, RR, LR, RL and
balancing using
subprograms and verify
it.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     int height;
7     struct Node *left;
8     struct Node *right;
9 };
10
11 int height(struct Node *root)
12 {
13     if (root == NULL)
14         return 0;
15     return root->height;
16 }
17
18 int max(int a, int b)
19 {
20     return (a > b) ? a : b;
```

3
30 20 10

```
153
154     printf("Inorder Traversal: ");
155     inorder(root);
156
157     printf("\nHeight: %d", height(root));
158
159     printf("\nAVL Tree is balanced.\n");
160
161     return 0;
162 }
```

OUTPUT

```
Enter number of nodes: Enter elements: Inorder Traversal: 10 20 30
Height: 2
AVL Tree is balanced.
```

AVL TREE - DELETION

Write a C program to delete a node 88 from an AVL Tree and rebalance the tree if necessary. Use the following elements 77, 22, 99, 55, 88, 33, 11, 66, 44.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     int height;
7     struct Node *left, *right;
8 };
9
10 int height(struct Node *n)
11 {
12     if (n == NULL)
13         return 0;
14     return n->height;
15 }
16
17 int max(int a, int b)
18 {
19     return (a > b) ? a : b;
20 }
```

```
221     root = insert(root, 11);
222     root = insert(root, 66);
223     root = insert(root, 44);
224
225     printf("Before Deletion: ");
226     inorder(root);
227
228     root = deleteNode(root, 88);
229
230     printf("\nAfter Deletion: ");
231     inorder(root);
232
233     return 0;
234 }
```

OUTPUT

```
Before Deletion: 11 22 33 44 55 66 77 88 99
After Deletion: 11 22 33 44 55 66 77 99
```


AVL TREE- HEIGHT

Write a C program to find the height of an AVL Tree. Create an AVL Tree 50 30 20 40 70 60. Display height after each insertion.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     int height;
7     struct Node *left, *right;
8 };
9
10 int height(struct Node *n)
11 {
12     if (n == NULL)
13         return 0;
14     return n->height;
15 }
16
17 int max(int a, int b)
18 {
19     return (a > b) ? a : b;
20 }
```

50 30 20 40 70 60

OUTPUT

```
After inserting 50: Height = 1
After inserting 30: Height = 2
After inserting 20: Height = 2
After inserting 40: Height = 3
After inserting 70: Height = 3
After inserting 60: Height = 3
Inorder Traversal: 20 30 40 50 60 70
```

AVL TREE- BALANCE FACTOR

Write a C program to display the balance factor of each node in an AVL Tree.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     struct Node *left, *right;
7 };
8
9 int height(struct Node *root)
10 {
11     if (root == NULL)
12         return 0;
13
14     int l = height(root->left);
15     int r = height(root->right);
16
17     return 1 + (l > r ? l : r);
18 }
19
20 struct Node* newNode(int data)
```

OUTPUT

Balance Factor of Each Node:

Node 20 : Balance Factor = 0

Node 30 : Balance Factor = 0

Node 40 : Balance Factor = 0

Node 50 : Balance Factor = 0

Node 60 : Balance Factor = 0

Node 70 : Balance Factor = 0

Node 80 : Balance Factor = 0

AVL Rotations with ▾

AVL ROTATIONS WITH BALANCING

Write a C program to demonstrate all four AVL rotations. LL Rotation, RR Rotation, LR Rotation, RL Rotation. Display AVL Tree before and after balancing.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     struct Node *left, *right;
7     int height;
8 };
9
10 int height(struct Node *n) {
11     return n ? n->height : 0;
12 }
13
14 int max(int a, int b) {
15     return (a > b) ? a : b;
16 }
17
18 struct Node* createNode(int data) {
19     struct Node *n = (struct Node*)malloc(sizeof(struct Node));
20     n->data = data;
```

OUTPUT

--- LL Rotation ---

Before balancing: 30 20 10

After balancing (Preorder): 20 10 30

--- RR Rotation ---

Before balancing: 10 20 30

After balancing (Preorder): 20 10 30

--- LR Rotation ---

Before balancing: 30 10 20